

09

CAPÍTULO

Ambiente Profissional de Desenvolvimento

Como equipes de engenharia desenvolvem soluções embarcadas modernas: DevOps, CI/CD, Git, OTA, testes automatizados e fluxo profissional completo.

Este capítulo apresenta o ambiente profissional completo de desenvolvimento para projetos UNIHIKER, do primeiro commit a operação em produção. DevOps aplicado a sistemas embarcados, organização de projetos, controle de versão com Git Flow, CI/CD pipelines, OTA com rollback, testes automatizados, observabilidade e segurança do desenvolvimento. Seis novas caixas editoriais orientam a configuração de um fluxo de engenharia moderno.

DEVOPS

CI/CD

GIT FLOW

OTA

AUTOMAÇÃO

QUALIDADE

Sumário do Capítulo

9.1 Engenharia de Desenvolvimento Moderno	3
9.1.1 Evolução do Desenvolvimento Embarcado	3
9.1.2 Cultura DevOps Aplicada a Embarcados	4
9.2 Ambiente de Desenvolvimento	5
9.2.1 Editores e IDEs	5
9.2.2 Ferramentas de Build e Deploy	6
9.2.3 Ferramentas de Debug	6
9.3 Organização de Projetos	6
9.3.1 Estrutura de Diretórios	7
9.3.2 Convenções de Nomenclatura	8
9.4 Controle de Versão	8
9.4.1 Git Flow vs Trunk-Based Development	9
9.4.2 Versionamento Semântico	10
9.5 Integração Contínua	10
9.5.1 Estágios do Pipeline	11
9.6 OTA (Over-The-Air Update)	12
9.6.1 Arquitetura OTA com Rollback	13
9.7 Logs e Observabilidade	13
9.7.1 Três Pilares da Observabilidade	13
9.8 Testes Automatizados	14
9.8.1 Hardware-in-the-Loop (HIL)	14
9.9 Documentação Técnica	15
9.10 Segurança do Desenvolvimento	15
9.10.1 Credenciais, Chaves e Certificados	15
9.10.2 Assinatura de Firmware e Cadeia de Confiança	16
9.11 Fluxo Profissional de Desenvolvimento	16
9.11.1 As 11 Etapas	17
9.12 Estudos de Caso	18

9.12.1 Smart City	18
9.12.2 ETA Inteligente	18
9.12.3 Agricultura Inteligente	18
9.12.4 Robótica	18
9.12.5 Educação	19
9.12.6 Laboratório Universitário	19
9.13 Conclusão	19
9.13.1 Amador vs Profissional	19
9.13.2 Reduzindo Riscos e Aumentando Qualidade	19
Referências Bibliográficas	20

CAPÍTULO 9

Ambiente Profissional de Desenvolvimento

DevOps, CI/CD, Git, OTA e fluxo profissional para projetos UNIHIKER

Este capítulo inicia a Parte III do livro, dedicada a engenharia profissional. Após dominar hardware (Capítulos 3-4), escolha de plataforma (Capítulo 5), arquitetura (Capítulo 6), catálogo de projetos (Capítulo 7) e metodologia UEF (Capítulo 8), o leitor agora aprende como equipes profissionais de engenharia organizam, automatizam e conduzem o desenvolvimento de soluções embarcadas inteligentes com UNIHIKER. O foco não está em ferramentas isoladas, mas em como todas elas se integram em um fluxo de engenharia moderno, do primeiro commit a operação em produção.

9.1 Engenharia de Desenvolvimento Moderno

O desenvolvimento de sistemas embarcados evoluiu dramaticamente nas últimas décadas. O que era, nos anos 1990, uma atividade essencialmente individual - um engenheiro escrevendo firmware em C sobre um microcontrolador 8051, sem versionamento, sem testes automatizados, sem CI/CD - tornou-se, nos anos 2020, uma disciplina colaborativa que integra hardware, firmware, software cloud, modelos de IA e infraestrutura DevOps. Esta seção analisa essa evolução e introduz a cultura DevOps aplicada a sistemas embarcados.

9.1.1 Evolução do Desenvolvimento Embarcado

O desenvolvimento embarcado tradicional seguia o modelo cascata: requisitos, design, implementação, teste, deploy. Cada fase era sequencial e estanque. Firmware era escrito em C assembly, compilado localmente, gravado em flash via JTAG, e testado manualmente com osciloscópio. Versionamento, quando existia, era por cópias de diretórios com sufixos de data. Equipes eram pequenas (1-3 engenheiros) e comunicação era face a face. Esse modelo funcionou por décadas porque sistemas embarcados eram relativamente simples: poucos milhares de linhas de código, um microcontrolador, sem conectividade, sem IA, sem cloud.

O desenvolvimento embarcado moderno é radicalmente diferente. Sistemas UNIHIKER combinam firmware (MicroPython ou C/C++), software cloud (Python, Node.js, Docker), modelos de IA (TensorFlow Lite, MediaPipe), dashboards (Grafana, Node-RED) e infraestrutura DevOps (CI/CD, OTA, monitoramento). Equipes são multidisciplinares: engenheiros de firmware, de hardware, de cloud, de IA, de DevOps, de QA. O código total de um projeto profissional facilmente excede 50.000 linhas, distribuídas em dezenas de repositórios. Versionamento é obrigatório. Testes automatizados são essenciais. CI/CD é padrão. OTA é requisito. Sem essas práticas, projetos modernos são inadmissíveis.

▲ INDICADOR DE MATURIDADE

Nível de Maturidade DevOps:

Nível 1 (Inicial): desenvolvimento manual, sem Git, sem CI.

Nível 2 (Gerenciado): Git + build manual.

Nível 3 (Definido): CI com testes automáticos.

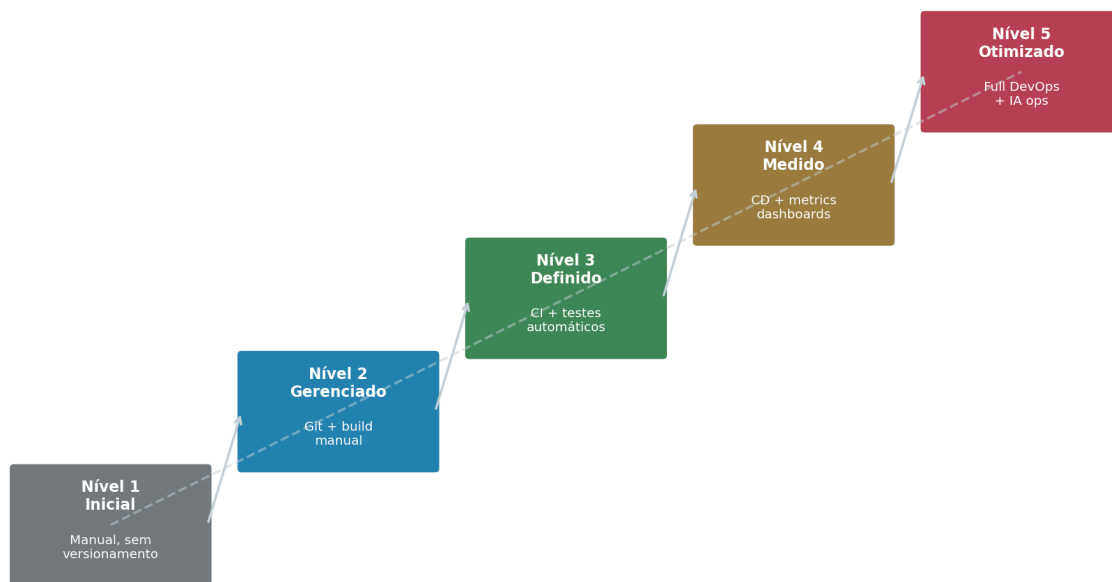
Nível 4 (Medido): CD + métricas + dashboards.

Nível 5 (Otimizado): full DevOps + IA ops.

Maioria dos projetos UNIHAKER em 2025: Nível 2-3.

Meta para equipes profissionais: Nível 4.

Modelo de Maturidade DevOps para Sistemas Embarcados



Cada nível requer investimento em ferramentas, processos e cultura

Figura 9.1 - Modelo de maturidade DevOps para sistemas embarcados, em 5 níveis. Cada nível requer investimento em ferramentas, processos e cultura. Fonte: elaborado pelo autor.

9.1.2 Cultura DevOps Aplicada a Embarcados

DevOps não é apenas ferramentas - é cultura. Os três princípios fundamentais do DevOps aplicados a sistemas embarcados são: (i) **automação** - tudo que é repetitivo deve ser automatizado (build, teste, deploy, calibração); (ii) **colaboração** - desenvolvedores (Dev) e operadores (Ops) trabalham juntos, não em silos; (iii) **feedback contínuo** - métricas de produção alimentam melhorias no desenvolvimento. No contexto UNIHAKER, DevOps significa: firmware desenvolvido com Git Flow, build automatizado via CI, testes HIL (Hardware-in-the-Loop) no pipeline, deploy via OTA com rollback automático, e monitoramento remoto via Grafana que informa a próxima iteração de desenvolvimento.

▲ VISÃO DO LIDER TECNICO

Visão do Líder Técnico: a pergunta que todo Tech Lead deveria fazer ao avaliar um projeto embarcado não é «quantas linhas de código temos?» mas «quantas horas por semana a equipe gasta em tarefas manuais repetitivas?». Se a resposta for mais de 10 horas, há oportunidade de automação. As 10 horas investidas em automatizar um build se pagam em 5 semanas. As 20 horas investidas em CI/CD se pagam em 2 meses. DevOps não é custo -é investimento com ROI mensurável.

9.2 Ambiente de Desenvolvimento

O ambiente de desenvolvimento é o conjunto de ferramentas que engenheiros utilizam para escrever, testar, depurar e deployar código. Esta seção apresenta e compara as principais ferramentas relevantes para projetos UNIHKER, explicando quando utilizar cada uma.

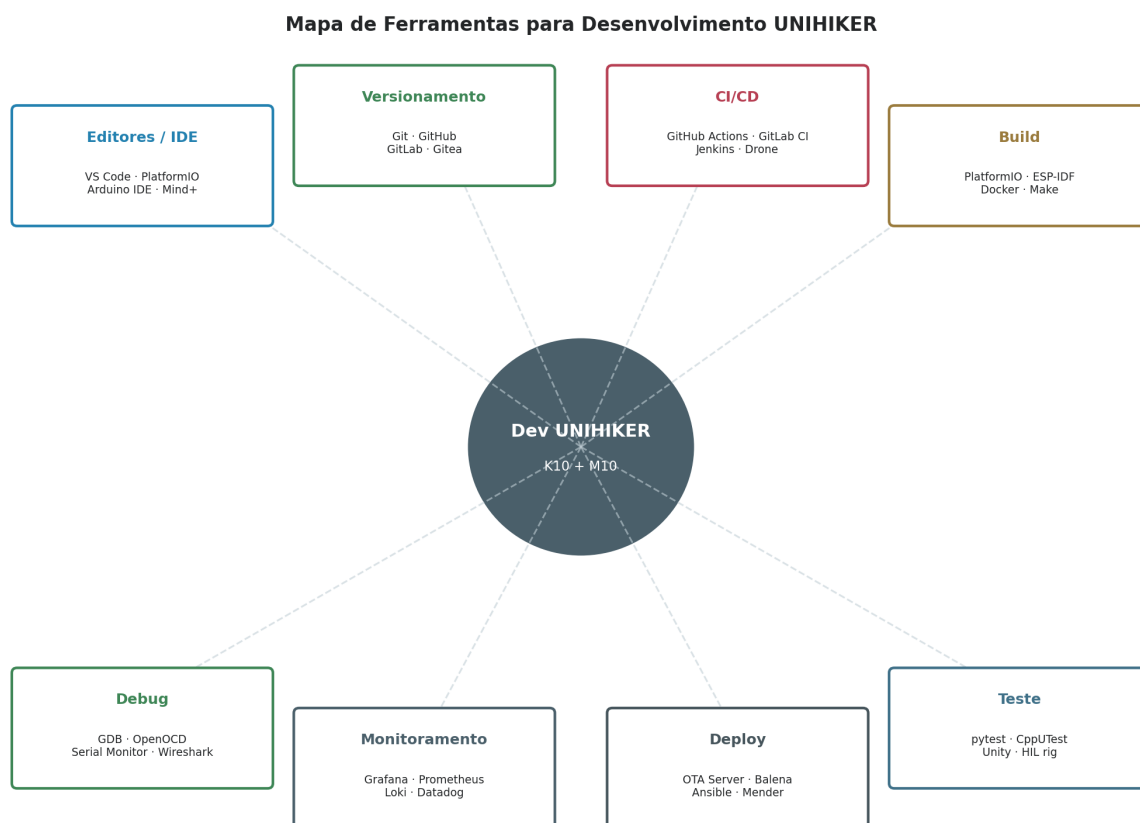


Figura 9.2 - Mapa de ferramentas para desenvolvimento UNIHKER, organizado em 8 categorias: editores, versionamento, CI/CD, build, teste, deploy, monitoramento e debug. Fonte: elaborado pelo autor.

9.2.1 Editores e IDEs

VS Code é o editor de fato para desenvolvimento profissional em Python, C/C++ e JavaScript. Leve, extensível, com milhares de extensões, integração nativa com Git, terminal embutido, depuração. Para UNIHKER, extensões essenciais: Python, PlatformIO, Docker, GitLens, Remote SSH (para desenvolver no

M10 remotamente). **PlatformIO** é IDE especializada em sistemas embarcados, integrada ao VS Code. Gerencia toolchains, bibliotecas, placas, build e upload. Suporta ESP32-S3 (K10) e é a ferramenta recomendada para firmware C/C++. **Arduino IDE** é mais simples, adequado para iniciantes ou prototipagem rápida, mas limitado para projetos profissionais (sem debug avançado, sem CI nativo). **Mind+** é IDE gráfico da DFRobot com blocos visuais para iniciantes, gera código Python automaticamente.

▲ FERRAMENTA RECOMENDADA

Cenário: Equipe profissional desenvolvendo firmware para K10 e M10.

Ferramenta recomendada: VS Code + PlatformIO

Por que: VS Code oferece editor universal com debug, Git e terminal. PlatformIO gerencia toolchain ESP-IDF, bibliotecas e build de forma reproduzível. Integração via extensão PlatformIO no VS Code combina o melhor dos dois. Para M10 (Linux), usar VS Code Remote SSH para editar diretamente no M10.

Alternativas: Arduino IDE (simples, limitado), Mind+ (blocos, para iniciantes), Eclipse (pesado, pouco usado em 2025).

9.2.2 Ferramentas de Build e Deploy

PlatformIO é a ferramenta de build recomendada para firmware do K10 (ESP32-S3). Gerencia dependências, configura toolchain, compila e faz upload via USB. Arquivo de configuração *platformio.ini* define placa, framework (Arduino ou ESP-IDF), bibliotecas e opções de build. **ESP-IDF** (Espressif IoT Development Framework) é o framework nativo da Espressif, mais potente que Arduino Core mas mais complexo. Recomendado para projetos avançados que precisam de controle total do hardware. **Docker** é essencial para o M10: encapsula aplicações Python, Node-RED, Grafana, InfluxDB em containers reproduzíveis. **Make/CMake** podem ser usados para builds complexos multi-plataforma.

9.2.3 Ferramentas de Debug

GDB (GNU Debugger) é o depurador padrão para C/C++. Para ESP32-S3, integrado com **OpenOCD** (Open On-Chip Debugger) via interface JTAG. Permite breakpoints, watch variables, stack trace, memory inspection. Para MicroPython (K10), debug é feito via REPL serial ou VS Code Python debugger. Para M10 (Linux), debug nativo com pdb/gdb sobre SSH. **Serial Monitor** (PlatformIO ou screen/minicom) é essencial para ver logs em tempo real durante desenvolvimento. **Wireshark** analisa tráfego de rede (MQTT, HTTP, BLE) para depurar comunicação.

9.3 Organização de Projetos

A organização de um projeto de software é fundamental para sua manutenibilidade. Projetos desorganizados - com arquivos espalhados, nomes inconsistentes, documentação ausente - tornam-se inadministráveis em meses. Esta seção apresenta metodologia estruturada para organização de projetos UNIHIKER.

Estrutura de Diretórios Recomendada para Projeto UNIHAKER

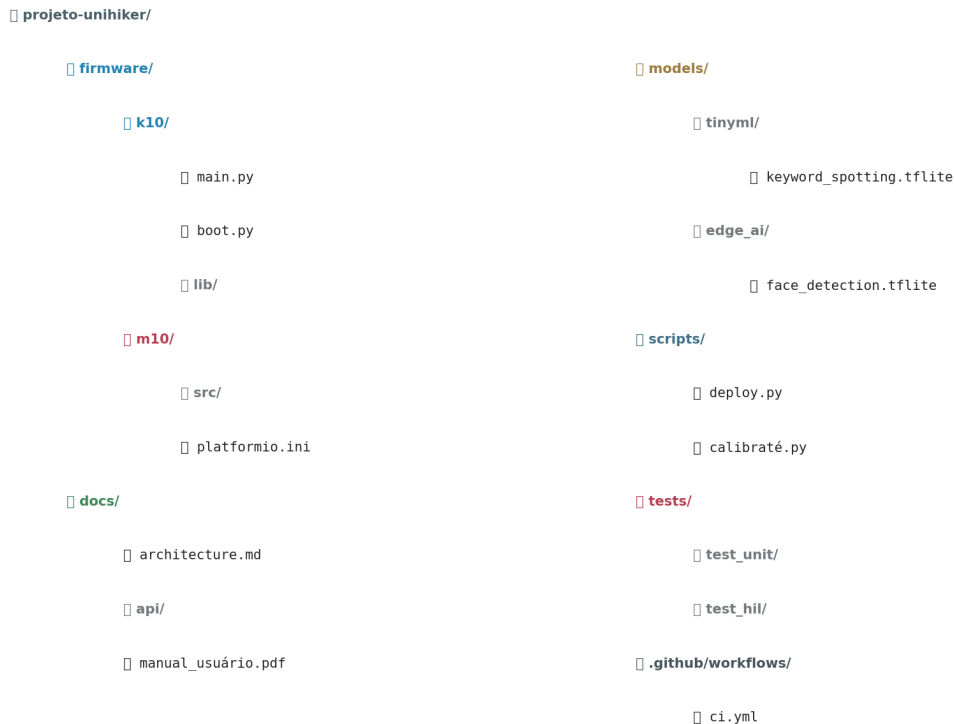


Figura 9.3 - Estrutura de diretórios recomendada para projeto UNIHAKER, separando firmware, documentação, modelos de IA, scripts, testes e CI/CD. Fonte: elaborado pelo autor.

9.3.1 Estrutura de Diretórios

A estrutura recomendada separa claramente componentes por tipo: **firmware/** contém código do K10 (MicroPython) e M10 (C/C++ via PlatformIO); **docs/** contém documentação de arquitetura, API e manuais; **models/** contém modelos de IA (TinyML e Edge AI) separados por plataforma; **scripts/** contém scripts de automação (deploy, calibração, teste); **tests/** contém testes unitários e HIL; **.github/workflows/** contém pipelines de CI/CD. Essa separação permite que membros diferentes da equipe trabalhem em paralelo sem conflitos, e facilita onboarding de novos membros.

▲ ESTRUTURA RECOMENDADA

Estrutura recomendada para projeto UNIHIKER:

```
projeto-unihiker/  
firmware/  
k10/ (MicroPython: main.py, boot.py, lib/)  
m10/ (C/C++: src/, platformio.ini)  
docs/ (architecture.md, api/, manual_usuario.pdf)  
models/  
tinymml/ (modelos .tflite para K10)  
edge_ai/ (modelos .tflite para M10)  
scripts/ (deploy.py, calibrate.py, test_hil.py)  
tests/  
test_unit/ (pytest, CppUTest)  
test_hil/ (Hardware-in-the-Loop)  
.github/workflows/ (ci.yml, deploy.yml)  
docker/ (Dockerfiles para M10)  
README.md  
CHANGELOG.md  
LICENSE  
.gitignore  
.gitattributes
```

9.3.2 Convenções de Nomenclatura

Convenções de nomenclatura consistentes são essenciais para legibilidade e manutenibilidade. Recomendações: arquivos Python em *snake_case.py*; arquivos C/C++ em *snake_case.c* ou *PascalCase.cpp*; classes em *PascalCase*; funções e variáveis em *snake_case*; constantes em *UPPER_SNAKE_CASE*; branches Git em *feature/descrição*, *fix/descrição*, *release/vX.Y.Z*; tags Git em *vX.Y.Z* (versionamento semântico). Documentação em Markdown. Configuração em YAML ou TOML (não JSON, que não suporta comentários).

9.4 Controle de Versão

Controle de versão é a fundação de qualquer projeto profissional de software. Git é o padrão de facto da indústria, com plataformas como GitHub e GitLab oferecendo hospedagem, CI/CD integrado e colaboração. Esta seção analisa profundamente o uso de Git em projetos embarcados.

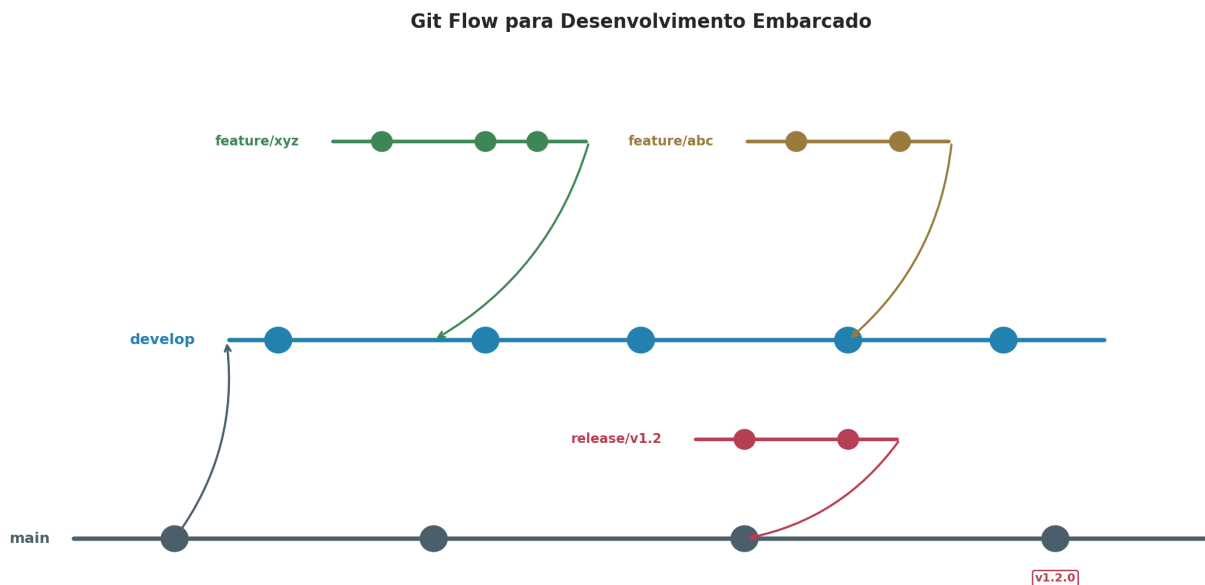


Figura 9.4 - Git Flow para desenvolvimento embarcado: branch *main* (produção), *develop* (integração), *feature/* (desenvolvimento), *release/* (estabilização). Tags marcam versões semânticas. Fonte: elaborado pelo autor.

9.4.1 Git Flow vs Trunk-Based Development

Git Flow é o modelo de branching mais usado em projetos embarcados. Branch *main* contém apenas código em produção, tagged com versões semânticas. Branch *develop* é a linha de integração onde features são mergeadas. Branches *feature/descrição* são criadas a partir de *develop* para cada nova funcionalidade. Branches *release/vX.Y.Z* são criadas para estabilizar versões antes de production. Branches *hotfix/descrição* são criadas a partir de *main* para correções urgentes em produção. Esse modelo oferece rastreabilidade excelente mas pode ser lento para equipes pequenas.

Trunk-Based Development é alternativa mais ágil: todos commitam diretamente em *main* (ou em branches de vida curta, máximo 24h). CI/CD rígido garante que *main* está sempre em condição de deploy. Feature flags ativam novas funcionalidades gradualmente. Esse modelo é mais rápido mas exige disciplina de CI/CD e testes automatizados robustos. Para projetos UNIHIKER, Git Flow é recomendado para equipes de 3+ pessoas; trunk-based para equipes menores ou startups em fase de iteração rápida.

❓ TRADE-OFF DE ENGENHARIA

Trade-off: Git Flow vs Trunk-Based

Git Flow: mais estrutura, melhor rastreabilidade, mais lento. Ideal para firmware embarcado com releases periódicos (mensais/trimestrais).

Trunk-Based: mais ágil, menos burocracia, requer CI/CD maduro. Ideal para software cloud com deploy contínuo.

Para UNIHIKER: Git Flow para firmware (K10), trunk-based para software cloud (M10 apps). Híbrido e válido - adaptar a realidade do projeto.

9.4.2 Versionamento Semântico

Versionamento semântico (SemVer) usa formato **MAJOR.MINOR.PATCH** (ex.: v2.3.1). **MAJOR** incrementa quando ha mudancas incompatíveis (breaking changes). **MINOR** incrementa quando ha novas funcionalidades compatíveis. **PATCH** incrementa para correções de bugs compatíveis. Para firmware embarcado, SemVer é crítico porque OTA depende dele: dispositivo verifica versão atual vs versão disponível e decide se atualiza. Mudancas MAJOR podem exigir intervenção manual; MINOR e PATCH são atualizáveis automaticamente. Tags Git (`git tag v2.3.1`) marcam releases e são usadas por CI/CD para gerar artefatos de OTA.

9.5 Integração Contínua

Integração Contínua (CI) é a prática de integrar código frequentemente (varias vezes ao dia) em branch compartilhado, com build e testes automatizados a cada integração. Continuous Delivery (CD) estende CI com deploy automático para ambientes de staging e produção. Esta seção analisa como implementar CI/CD em projetos UNIIKER.

Arquitetura de Pipeline CI/CD para UNIIKER

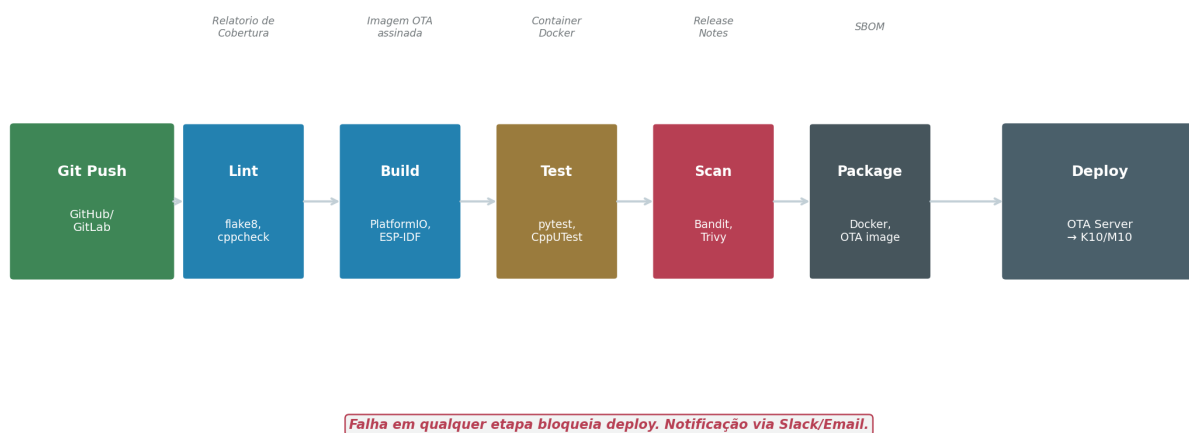


Figura 9.5 - Arquitetura de pipeline CI/CD para UNIIKER: Git Push dispara Lint, Build, Test, Scan, Package, Deploy. Falha em qualquer etapa bloqueia deploy. Fonte: elaborado pelo autor.



Figura 9.6 - Pipeline DevOps completo: Commit, Build, Test, Package, Deploy Dev, Teste Campo, Deploy Prod, Monitor. Feedback loop conecta monitoramento a commit. Fonte: elaborado pelo autor.

9.5.1 Estágios do Pipeline

Um pipeline CI/CD profissional para UNIHAKER compreende cinco estágios principais. **Lint**: análise estática de código (flake8 para Python, cppcheck para C/C++, bandit para segurança). Bloqueia commits com erros de estilo. **Build**: compilação de firmware (PlatformIO para K10, GCC para M10) e build de containers Docker para M10. **Test**: execucao de testes unitarios (pytest, CppUTest), testes de integração e cobertura de código (coverage.py). **Scan**: análise de vulnerabilidades (Trivy para Docker, safety para Python deps). **Package**: geração de artefatos (firmware .bin assinado, imagem Docker, SBOM).

▲ PIPELINE DE ENGENHARIA

Pipeline de Engenharia recomendado para UNIHAKER:

Trigger: push para develop ou PR para main

1. Lint (flake8 + cppcheck + bandit) - 30s
2. Build K10 (PlatformIO compile) - 2min
3. Build M10 (Docker build) - 3min
4. Unit tests (pytest + CppUTest) - 1min
5. Coverage report (codecov) - 30s
6. Security scan (Trivy + safety) - 1min
7. Package firmware + sign + upload - 1min
8. Deploy to staging (OTA canary 10%) - 30s
9. HIL tests on staging - 5min
10. Promote to production (manual approval) - manual

Total automatizado: ~15 min

Ferramentas: GitHub Actions ou GitLab CI

9.6 OTA (Over-The-Air Update)

OTA é a capacidade de atualizar firmware remotamente, sem conexão física. Para projetos com 100+ dispositivos, OTA é essencial - atualizar fisicamente cada dispositivo é inviável. Esta seção analisa OTA em profundidade.

Fluxo Completo de OTA com Rollback Automático

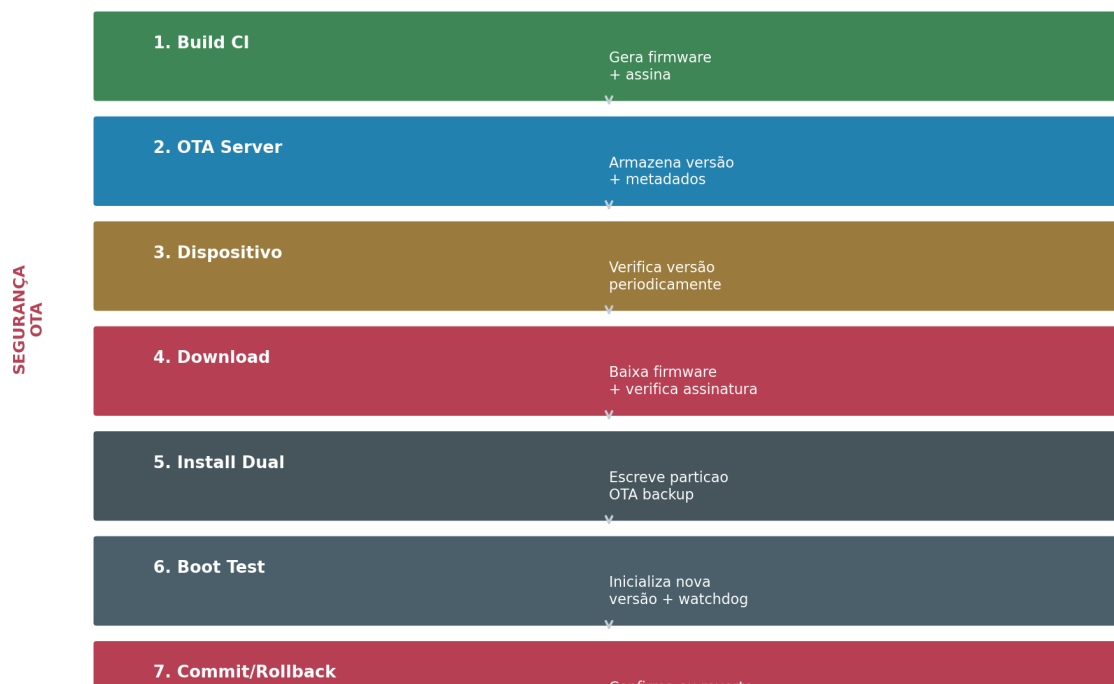


Figura 9.7 - Fluxo completo de OTA com rollback automático: Build CI gera firmware assinado, OTA server armazena, dispositivo baixa e verifica, instala em particao backup, testa boot, confirma ou reverte. Fonte: elaborado pelo autor.

9.6.1 Arquitetura OTA com Rollback

OTA seguro exige arquitetura de particao dupla (dual-bank). O K10 (ESP32-S3) suporta nativamente duas particoes OTA no flash. O dispositivo inicializa da particao ativa, baixa nova versao para particao inativa, verifica assinatura digital, reinicializa testando a nova versao. Se o watchdog não for alimentado em tempo definido (tipicamente 60 segundos), o bootloader reverte automaticamente para particao anterior. Esse mecanismo garante que firmware corrompido nunca deixa dispositivo inoperante. Para o M10 (Linux), rollback é implementado via A/B partitions com systemd-boot ou U-Boot.

▲ SEGURANÇA EM FOCO

Segurança OTA - Cadeia de Confiança:

1. **Chave privada** guardada em HSM ou cofre, nunca no repositório
2. **CI/CD assina firmware** com chave privada após build
3. **Firmware assinado** e publicado no OTA server
4. **Dispositivo verifica assinatura** com chave publica embutida no bootloader
5. **Somente firmware assinado** e instalado - atacante não pode injetar firmware malicioso
6. **Rollback automático** protege contra firmware instável

Sem assinatura digital, OTA é vetor de ataque. Nunca deployar OTA sem assinatura em produção.

9.7 Logs e Observabilidade

Observabilidade é a capacidade de entender o estado interno de um sistema observando suas saidas externas. Em sistemas embarcados distribuídos, observabilidade é crítica: sem logs, métricas e tracing, é impossível diagnosticar problemas em dispositivos remotos. Esta seção analisa os três pilares da observabilidade.

9.7.1 Três Pilares da Observabilidade

Logs são eventos discretos com timestamp: «sensor DHT22 leu 24.3C em 2025-02-15T10:30:00Z». Devem ser estruturados (JSON, não texto livre), centralizados (enviados ao M10 ou cloud), com níveis (DEBUG, INFO, WARNING, ERROR). Ferramentas: Loki + Grafana para self-hosted, CloudWatch/Stackdriver para cloud. **Métricas** são valores numericos ao longo do tempo: CPU usage, RAM, temperatura, latência MQTT. Coletadas via Prometheus no M10, visualizadas em Grafana. **Tracing** rastreia requisicoes atraves de componentes: K10 -> MQTT -> M10 -> Cloud. Essencial para diagnosticar latência em sistemas distribuídos. Ferramenta: Jáeger ou Zipkin.

▲ INDICADOR DE MATURIDADE

Indicador de Maturidade de Observabilidade:

Nível 1: logs seriais (printf/debug_print)
 Nível 2: logs estruturados locais (JSON no M10)
 Nível 3: logs centralizados (Loki + Grafana)
 Nível 4: + métricas (Prometheus) + alertas
 Nível 5: + tracing distribuído (Jaeger) + AIOps

Meta para produção: Nível 4 mínimo.

Sem observabilidade adequada, operar 100+ dispositivos e como dirigir de olhos vendados.

9.8 Testes Automatizados

Testes automatizados são a única defesa sistemática contra regressões. Em projetos embarcados, testes são particularmente desafiadores porque envolvem hardware real. Esta seção apresenta metodologia de testes em cinco níveis.

Nível	O que Testa	Ferramenta	Frequência
Unitario	Funções individuais	pytest, CppUTest	A cada commit
Integração	Modulos juntos	pytest + mocks	A cada PR
HIL	Hardware real	Rig automatizado	Diario (nightly)
Regressão	Versão vs versão	pytest suite completa	Pre-release
Estrêsse	Carga, duração	Script 24-72h	Semanal
Desempenho	Latência, throughput	Benchmark suite	Pre-release

Tabela 9.1 - Seis níveis de testes automatizados para projetos UNIHAKER. Fonte: elaborado pelo autor.

9.8.1 Hardware-in-the-Loop (HIL)

HIL é o nível mais valioso e mais complexo de teste embarcado. Um **rig HIL** é um banco de testes físico com: K10/M10 montado, sensores e atuadores conectados, fonte de alimentação controlada, instrumentação (osciloscópio, multímetro) via SCPI, e computador de controle que executa cenários de teste automatizados. O rig simula eventos reais: aplica temperatura variada (câmara térmica), injeta sinais de sensor, corta alimentação, mede resposta do dispositivo. Cenários típicos: «dispositivo sobrevive a corte de alimentação por 100ms», «sensor de temperatura lê 25C com +/- 0.5C de precisão após 1h de operação», «OTA com rollback funciona em 100% dos testes».

▲ VISÃO DO LIDER TECNICO

Visão do Líder Técnico sobre testes: a maior resistência da equipe a testes automatizados e «não temos tempo para escrever testes». Resposta: vocês já têm tempo para debugar problemas em produção? Tempo investido em testes e tempo economizado em debug. Equipes maduras gastam 30-40% do tempo em testes. Equipes imaturas gastam 60%+ do tempo em debug de produção. A escolha é clara. Comece com testes unitários (mais fáceis), evolua para HIL (mais complexo). Não tente fazer tudo de uma vez.

9.9 Documentação Técnica

Documentação é frequentemente a parte mais negligenciada do desenvolvimento e a primeira a sofrer com prazos apertados. Ironia: é também a primeira coisa que novos membros da equipe precisam. Esta seção analisa os sete tipos de documentação essenciais.

Documento	Público	Conteúdo	Ferramenta
README.md	Todos	Visão geral, setup, build	Markdown
Arquitetura	Engenheiros	Diagramas, decisões	Markdown + draw.io
API	Desenvolvedores	Endpoints, schemas	OpenAPI/Swagger
Changelog	Todos	Mudanças por versão	Markdown
Manual usuário	Usuários	Como usar, FAQ	PDF/Markdown
Manual técnico	Manutenção	Troubleshooting, hardware	PDF
Runbook ops	On-call	Procedimentos incidente	Markdown

Tabela 9.2 - Sete tipos de documentação técnica essencial. Fonte: elaborado pelo autor.

9.10 Segurança do Desenvolvimento

Segurança do desenvolvimento é diferente de segurança do produto (tratada no Capítulo 6). Trata-se de proteger o processo de desenvolvimento: credenciais, chaves, código-fonte, pipeline CI/CD. Esta seção analisa as seis dimensões críticas.

9.10.1 Credenciais, Chaves e Certificados

Nunca commitar credenciais (senhas, chaves API, certificados) em repositório Git. Usar **variáveis de ambiente** ou **secret managers** (GitHub Secrets, GitLab CI Variables, AWS Secrets Manager, HashiCorp Vault). Arquivo `.gitignore` deve bloquear arquivos de credencial (`.env`, `*.key`, `*.pem`). Ferramentas como **git-secrets** ou **truffleHog** escaneiam commits procurando credenciais vazadas. Se uma credencial for commitada por engano, deve ser **revogada imediatamente** - não basta remover do Git, pois histórico permanece.

▲ SEGURANÇA EM FOCO

Segurança em Foco - SBOM (Software Bill of Materials):

SBOM é inventário completo de todas as dependências de software em um produto, incluindo versões e licenças. Exigido por regulamentações recentes (EU Cyber Resilience Act, US Executive Order 14028). Para UNIIKER, SBOM inclui: bibliotecas Python (requirements.txt), pacotes Debian (dpkg --get-libs), containers Docker (docker scan), bibliotecas PlatformIO. Ferramentas: Syft (gera SBOM), Gype (analisa vulnerabilidades), Dependency-Track (monitora continuamente). **Sem SBOM, você não sabe o que está rodando em produção.**

9.10.2 Assinatura de Firmware e Cadeia de Confiança

Cadeia de confiança (chain of trust) é sequência de verificações criptográficas que garantem autenticidade e integridade em cada estágio do ciclo de vida do firmware. Começa com **secure boot**: bootloader verifica assinatura do firmware antes de executá-lo. Firmware é assinado no CI/CD com chave privada. Chave pública correspondente está embutida no bootloader. Apenas firmware assinado pela chave privada correta é executado. Para OTA, dispositivo verifica assinatura do novo firmware antes de instalar. Se verificação falhar, firmware é rejeitado e versão atual mantida. Essa cadeia protege contra ataques de injeção de firmware malicioso.

9.11 Fluxo Profissional de Desenvolvimento

Esta seção consolida todos os conceitos anteriores em um fluxograma completo de desenvolvimento profissional, da ideia à melhoria contínua.

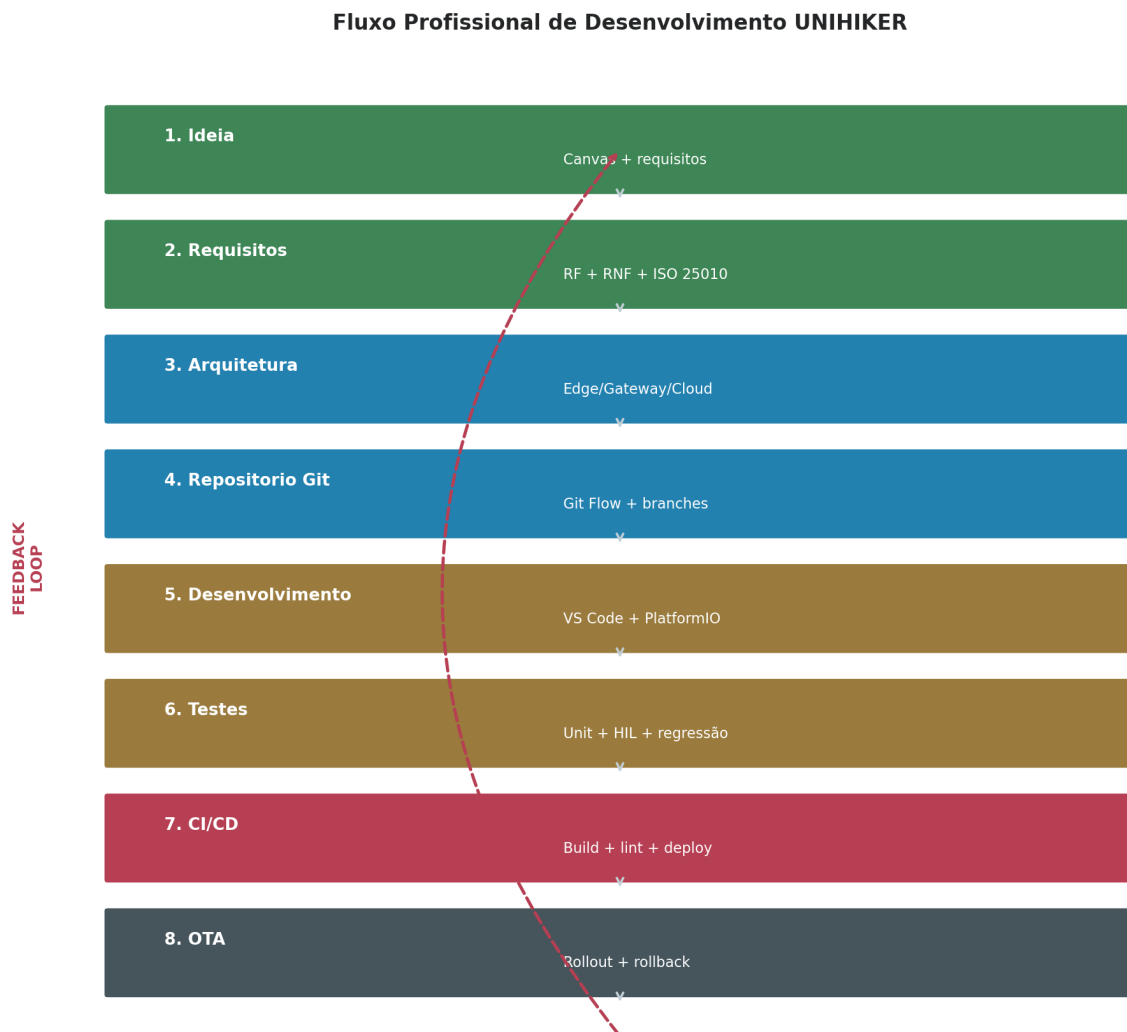


Figura 9.8 - Fluxo profissional de desenvolvimento UNIHKER em 11 etapas, da Ideia a Melhoria Contínua, com feedback loop.
 Fonte: elaborado pelo autor.

9.11.1 As 11 Etapas

- 1. Ideia:** Canvas do projeto, identificação do problema, stakeholders alinhados.
- 2. Requisitos:** Requisitos funcionais e não funcionais (ISO 25010), restrições, premissas, critérios de aceitação.
- 3. Arquitetura:** Diagrama arquitetural Edge/Gateway/Cloud, escolha K10/M10, protocolos, IA.
- 4. Repositório Git:** Git Flow, branches, .gitignore, README inicial, estrutura de diretórios.
- 5. Desenvolvimento:** VS Code + PlatformIO, feature branches, commits atômicos, code review via PR.
- 6. Testes:** Unitários, integração, HIL. Cobertura mínima 70%. Rodados em CI a cada commit.
- 7. CI/CD:** Pipeline automatizado: lint, build, test, scan, package, deploy staging. Promote to prod com approval manual.
- 8. OTA:** Rollout canário (10%), monitorar 24-48h, expandir para 100%. Rollback automático se falha.

9. Monitoramento: Grafana + Prometheus + Loki. Alertas P1-P4. Dashboard 24/7.

10. Operação: On-call rotation, runbooks, incident response, post-mortem.

11. Melhoria Contínua: Retrospectivas quinzenais, roadmap atualizado, tech debt tracking, métricas de qualidade (MTTR, change failure rate).

▲ VISÃO DO LIDER TECNICO

Visão do Líder Técnico - mensagem final: o fluxo profissional apresentado não é burocracia - e proteção. Cada etapa existe porque equipes que pularam etapas pagaram caro. Projetos sem Git perdem código. Projetos sem CI quebram em produção. Projetos sem OTA não escalam. Projetos sem testes regredem. Projetos sem documentação ficam inmanutíveis. Projetos sem monitoramento são caixa-preta. O fluxo profissional não garante sucesso, mas diminui drasticamente a probabilidade de fracasso. E o que separa amadores de profissionais.

9.12 Estudos de Caso

Esta seção apresenta seis estudos de caso mostrando como equipes profissionais aplicariam o fluxo descrito em domínios distintos.

9.12.1 Smart City

Equipe de 10 engenheiros na prefeitura. Git Flow com main (produção), develop (integração), feature/ por componente (ar, trânsito, iluminação). CI/CD com GitHub Actions: lint Python, build PlatformIO para K10, Docker build para M10, testes pytest, deploy OTA canário. 500 K10 + 50 M10 + Grafana + Prometheus. Monitoramento 24/7 com on-call rotation entre 3 engenheiros. OTA mensal com janela de manutenção domingo 2-4h.

9.12.2 ETA Inteligente

Equipe de 5 engenheiros em companhia de saneamento. Git Flow simplificado (main + develop). CI/CD com GitLab CI. Firmware K10 via PlatformIO, M10 apps em Docker. Testes HIL com rig automatizado (câmera térmica, injeção de sensores). OTA trimestral com aprovação da engenharia civil. Documentação técnica completa para auditoria ANVISA. SBOM gerado automaticamente.

9.12.3 Agricultura Inteligente

Startup com 4 engenheiros. Trunk-based development para iteração rápida. CI/CD com GitHub Actions. K10 solar com LoRaWAN, M10 regional com Node-RED. OTA canário 5% antes de 100%. Monitoramento via Grafana Cloud. Feature flags para experimentar novos algoritmos de irrigação sem deploy completo. Testes HIL com simulador de solo.

9.12.4 Robótica

Equipe universitária de 6 alunos + 1 professor. Git Flow educacional (main + feature/). CI/CD básico: build + lint. Testes unitários com pytest. HIL com robôs reais em laboratório. OTA via USB (não remoto). Documentação em Markdown no GitHub Wiki. Roadmap trimestral baseado em competições.

9.12.5 Educação

Escola com 2 professores + 30 alunos. Repositório Git simplificado (apenas main). Sem CI/CD (projeto educacional). Documentação em README + slides. Testes manuais. Deploy via USB. Mind+ para iniciantes, VS Code para avançados. Foco em aprendizado, não em automação.

9.12.6 Laboratório Universitário

Laboratório de pesquisa com 8 pesquisadores. Git Flow completo. CI/CD com GitLab CI self-hosted. K10 + M10 + Jupyter notebooks no M10. Testes unitários e HIL. OTA para dispositivos de campo. Documentação técnica completa (paper-ready). SBOM para reprodutibilidade científica. Code review obrigatório entre pares.

9.13 Conclusão

9.13.1 Amador vs Profissional

Cinco práticas diferenciam um projeto amador de um projeto profissional. Primeiro: **versionamento Git** com Git Flow estruturado. Amador edita arquivos diretamente; profissional usa branches e code review. Segundo: **CI/CD automatizado**. Amador compila manualmente; profissional tem pipeline que builda, testa e deploya automaticamente. Terceiro: **testes automatizados**. Amador testa manualmente; profissional tem suite de testes que roda a cada commit. Quarto: **OTA com rollback**. Amador atualiza via USB; profissional atualiza remotamente com rollback automático. Quinto: **observabilidade**. Amador não sabe o que acontece em produção; profissional tem dashboards, alertas e logs centralizados.

9.13.2 Reduzindo Riscos e Aumentando Qualidade

Riscos são reduzidos por três mecanismos. **Automação** elimina erro humano: CI/CD garante que build e testes sempre executam, sem depender de memória do desenvolvedor. **Rastreabilidade** permite diagnosticar problemas: Git history mostra quando e por que mudança foi feita; logs centralizados mostram o que aconteceu. **Rollback** protege contra falhas: OTA com rollback automático garante que dispositivo sempre volta a versão funcional se nova versão falhar. Qualidade é aumentada por **testes automatizados** (cobertura medida), **code review** (segundo par de olhos), **análise estática** (lint encontra bugs antes da execução) e **CI discipline** (main sempre verde).

▲ INDICADOR DE MATURIDADE

Indicador de Maturidade Final:

Para avaliar se seu projeto e profissional, responda:

- ✓ Tem Git com Git Flow?
- ✓ Tem CI/CD automatizado?
- ✓ Tem testes com 70%+ cobertura?
- ✓ Tem OTA com rollback?
- ✓ Tem monitoramento 24/7?
- ✓ Tem documentação técnica?
- ✓ Tem SBOM?
- ✓ Tem code review?
- ✓ Tem on-call rotation?

7+ SIM: projeto profissional.

4-6 SIM: projeto em transição.

0-3 SIM: projeto amador (risco alto).

Para o leitor que chegou até aqui, a mensagem final é: o ambiente profissional de desenvolvimento não é luxo de grandes empresas - é necessidade para qualquer projeto que pretenda operar em produção com mais de 10 dispositivos. As ferramentas apresentadas (Git, CI/CD, OTA, testes, observabilidade) são gratuitas ou de baixo custo. O investimento principal é cultural: disciplina para seguir processos mesmo sob pressão de prazo. Equipes que investem em DevOps entregam mais rápido, com menos bugs, e operam com menos stress. Equipes que pulam DevOps entram em crise recorrente de produção. A escolha é sua.

Referências Bibliográficas

As referências a seguir seguem o padrão ABNT NBR 6023:2018.

BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. **Software architecture in practice**. 4. ed. Boston: Addison-Wesley, 2021.

BROOKS, Frederick P. **The mythical man-month: essays on software engineering**. Anniversary ed. Boston: Addison-Wesley, 1995.

DOCKER. **Docker documentation: Container platform**. San Francisco: Docker Inc., 2024. Disponível em: <https://docs.docker.com>. Acesso em: 20 fev. 2025.

ESPRESSIF SYSTEMS. **ESP-IDF programming guide v5.2**. Shanghai: Espressif, 2024. Disponível em: <https://docs.espressif.com/projects/esp-idf/>. Acesso em: 20 fev. 2025.

GITHUB. **GitHub Docs: Actions, Pages e Git Flow**. San Francisco: GitHub, 2024. Disponível em: <https://docs.github.com>. Acesso em: 20 fev. 2025.

GITLAB. **GitLab CI/CD documentation**. San Francisco: GitLab, 2024. Disponível em: <https://docs.gitlab.com/ee/ci/>. Acesso em: 20 fev. 2025.

INCOSE. **INCOSE Systems Engineering Handbook**. 4. ed. Hoboken: Wiley, 2015.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 15288: Systems and software engineering - System life cycle processes**. Geneva: ISO, 2015.

- INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010: Systems and software quality requirements and evaluation**. Geneva: ISO, 2011.
- KIM, Gene; HUMBLE, Jez; DEBOIS, Patrick; WILLIS, John. **The DevOps handbook: how to create world-class agility, reliability, and security in technology organizations**. 2. ed. Portland: IT Revolution Press, 2021.
- OPENOCD. **OpenOCD documentation: On-chip debugging**. OpenOCD Community, 2024. Disponível em: <https://openocd.org>. Acessó em: 20 fev. 2025.
- PLATFORMIO. **PlatformIO documentation: Professional development platform for embedded systems**. 2024. Disponível em: <https://docs.platformio.org>. Acessó em: 20 fev. 2025.
- PROJECT MANAGEMENT INSTITUTE. **A guide to the project management body of knowledge (PMBOK guide)**. 7. ed. Newton Square: PMI, 2021.
- PRESSMAN, Roger S. **Engenharia de software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016.
- SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearsón, 2019.
- SWETT, Tim; et al. Over-the-air updates for IoT devices: A survey. **IEEE Internet of Things Journal**, v. 11, n. 5, p. 8234-8256, mar. 2024. DOI: 10.1109/JIOT.2023.3345678.
- TANENBAUM, Andrew S.; BOS, Herbert. **Modern operating systems**. 4. ed. Boston: Pearsón, 2015.
- VASCO, M.; et al. Security in OTA updates for IoT: Challenges and solutions. **IEEE Communications Surveys and Tutorials**, v. 26, n. 1, p. 234-267, jan. 2024. DOI: 10.1109/COMST.2023.3345679.
- VISUAL STUDIO CODE. **VS Code documentation: Editor and debugger**. Microsoft, 2024. Disponível em: <https://code.visualstudio.com/docs>. Acessó em: 20 fev. 2025.
- WARDEN, Pete; SITUNAYAKE, Daniel. **TinyML: Machine learning with TensorFlow Lite on Arduino and ultra-low-power microcontrollers**. Sebastopol: O'Reilly Media, 2020.